

# Cloud Computing Lab 1 Report

Huizhe Su, Arihant Jain, Jordy, Nizar

February 2026

## Contents

|          |                                                                                                                |          |
|----------|----------------------------------------------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Server Installation</b>                                                                                     | <b>2</b> |
| 1.1      | Install Ubuntu 18.04 LTS in Amazon EC2 . . . . .                                                               | 2        |
| <b>2</b> | <b>Installing Apache Web Server</b>                                                                            | <b>2</b> |
| 2.4      | Install the Apache 2 HTTP Server . . . . .                                                                     | 2        |
| 2.5      | Check the Status of Apache 2 and Give Its Version . . . . .                                                    | 2        |
| 2.6      | Create a Home Page and Test with a Web Browser . . . . .                                                       | 3        |
| 2.7      | Create a DNS Zone with CloudDNS. Give the CNAME and ALIAS Created . . . . .                                    | 4        |
| 2.8      | Check DNS Resolution with the dig Command . . . . .                                                            | 4        |
| 2.9      | Test with a Web Browser . . . . .                                                                              | 5        |
| 2.10     | Create and Install a Self-Signed Certificate. Test with a Web Browser . . . . .                                | 5        |
| 2.11     | Create and Install a Certified Certificate with ZeroSSL. Test with a Web Browser . . . . .                     | 6        |
| 2.12     | Create and Install a Certified Certificate with Let's Encrypt and Certbot. Test with a Web Browser . . . . .   | 6        |
| 2.13     | Enable the Apache Rewrite Engine and Create the HTTP to HTTPS Redirect Rule. Test with a Web Browser . . . . . | 7        |
| <b>3</b> | <b>Creating a PHP Application</b>                                                                              | <b>8</b> |
| 3.14     | Install the PHP Module . . . . .                                                                               | 8        |
| 3.15     | Create a phpinfo.php Script and Identify the PHP Version . . . . .                                             | 8        |
| 3.16     | Create a meteo.php Script That Displays Weather Information Using OpenWeather . . . . .                        | 9        |

## Exercise 1. Server Installation

### Question 1. Install Ubuntu 18.04 LTS in Amazon EC2

After registering on AWS and setting up an EC2 instance, we have the following information page.

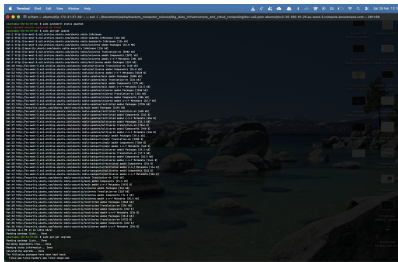


Figure 1: Huize Su

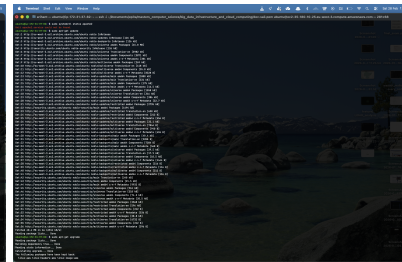


Figure 2: Other Student

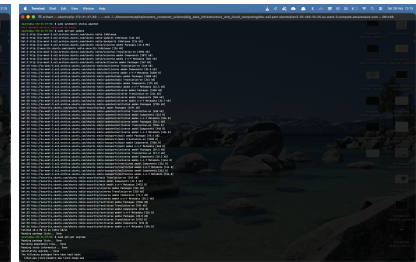


Figure 3: Other Student

## Exercise 2. Installing Apache Web Server

### Question 4. Install the Apache 2 HTTP Server

To serve web content from our EC2 instance, we install the Apache2 web server and enable the service so that it starts automatically on boot.

```
sudo apt install apache2
sudo systemctl start apache2
sudo systemctl enable apache2
```

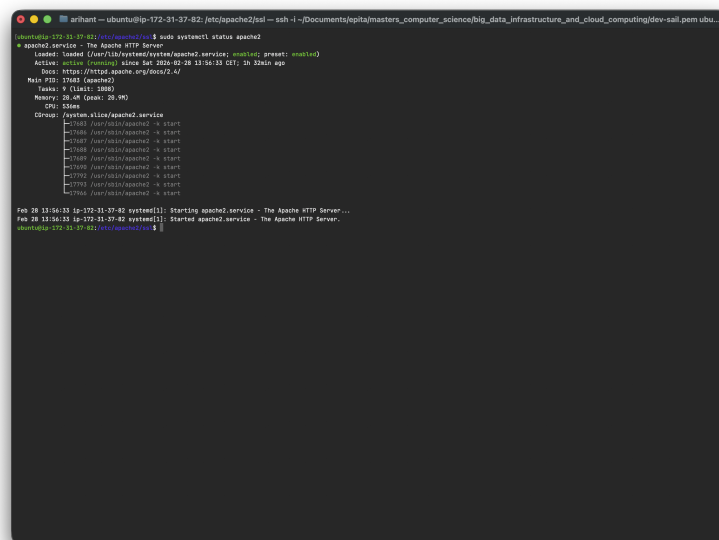


Figure 4: Apache2 service running successfully (Arihant Jain)

### Question 5. Check the Status of Apache 2 and Give Its Version

We can verify that Apache is running correctly and check its version:

```
sudo systemctl status apache2
apache2 -v
```

The `systemctl status` command confirms that the service is **active (running)**. The `apache2 -v` command outputs the installed version:

```
Server version: Apache/2.4.58 (Ubuntu)
Server built: 2025-12-09T15:50:28
```

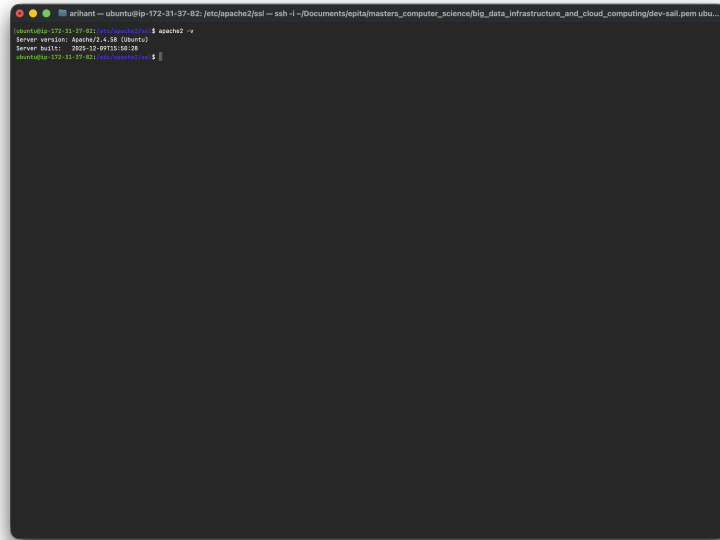


Figure 5: Apache2 version output (Arihant Jain)

#### Question 6. Create a Home Page and Test with a Web Browser

With Apache running, the default page is accessible at:

<http://ec2-35-180-10-25.eu-west-3.compute.amazonaws.com>

This serves the default Apache2 Ubuntu page located at `/var/www/html/index.html`. We then edited this file to serve our own content:

```
sudo nano /var/www/html/index.html
```

We replaced the default content with a simple custom page:

```
<html>
  <head>
    <title>My Server</title>
  </head>
  <body>
    Bienvenue sur mon serveur
  </body>
</html>
```

After saving the file, refreshing the browser shows our custom page.



Figure 6: Custom home page: “Bienvenue sur mon serveur” (Arihant Jain)

### Question 7. Create a DNS Zone with CloudDNS. Give the CNAME and ALIAS Created

To access our server using a custom domain name instead of the long EC2 public DNS, we set up a free DNS zone on CloudDNS:

1. Navigate to <https://www.cloudns.net>
2. Go to **Create a Zone** → **Free Zone**
3. Choose the name **arihant-dev-sail** with the suffix **.cloud-ip.cc**

We then created the following DNS records:

- **CNAME record:** **arihant-dev-sail.cloud-ip.cc** → **ec2-35-180-10-25.eu-west-3.compute.amazonaws.com**
- **A record (ALIAS):** **arihant-dev-sail.cloud-ip.cc** → **35.180.10.25**

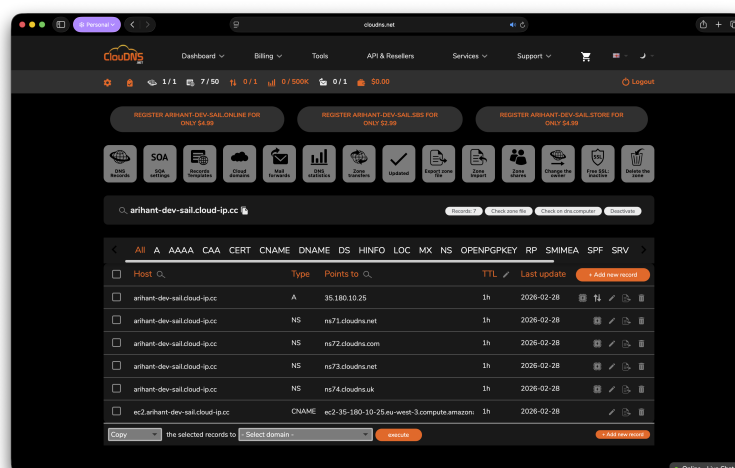


Figure 7: CloudDNS zone configuration with CNAME and A record (Arihant Jain)

### Question 8. Check DNS Resolution with the dig Command

After DNS propagation, we verify that the domain resolves correctly using the **dig** command:

```
dig +short ec2.arihant-dev-sail.cloud-ip.cc
```



The output confirms the A record resolving to our EC2 public IPv4 address 35.180.10.25, confirming that the DNS zone is properly configured.

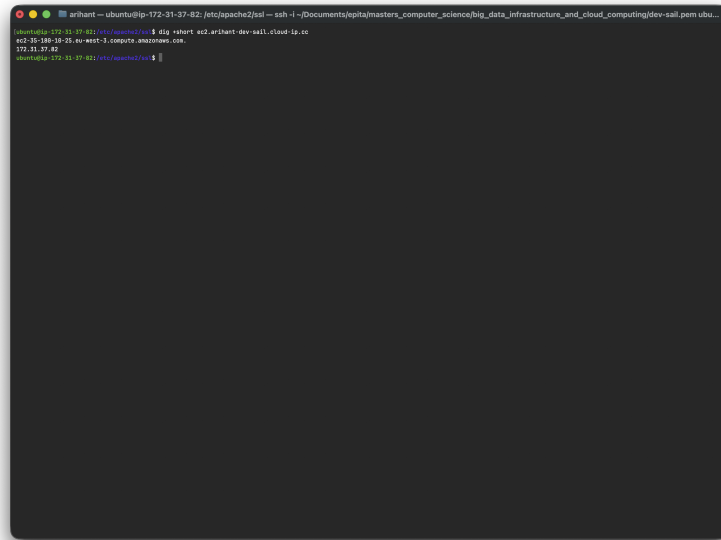


Figure 8: DNS resolution verified with dig (Arihant Jain)

#### Question 9. Test with a Web Browser

After DNS propagation, the server is now accessible via our custom domain at:

`http://arihant-dev-sail.cloud-ip.cc`

The browser displays our custom “Bienvenue sur mon serveur” page, confirming that the DNS is correctly pointing to our EC2 instance. However, the connection is not secured — the site is served over plain HTTP, as shown by the browser warning.

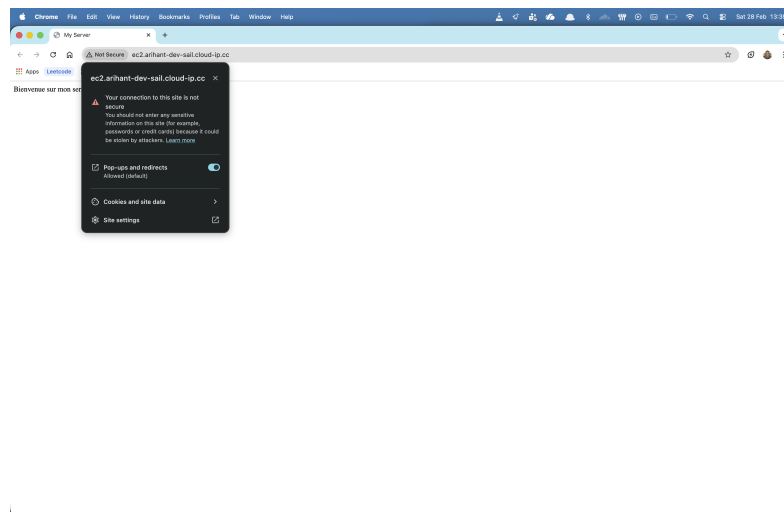


Figure 9: Site accessible via custom domain over HTTP (Arihant Jain)

#### Question 10. Create and Install a Self-Signed Certificate. Test with a Web Browser

As a first step toward securing the site, we generated a self-signed SSL certificate. We enabled the SSL module in Apache and configured it:

```
sudo a2enmod ssl
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
```

```

-keyout /etc/ssl/private/apache-selfsigned.key \
-out /etc/ssl/certs/apache-selfsigned.crt
sudo systemctl restart apache2

```

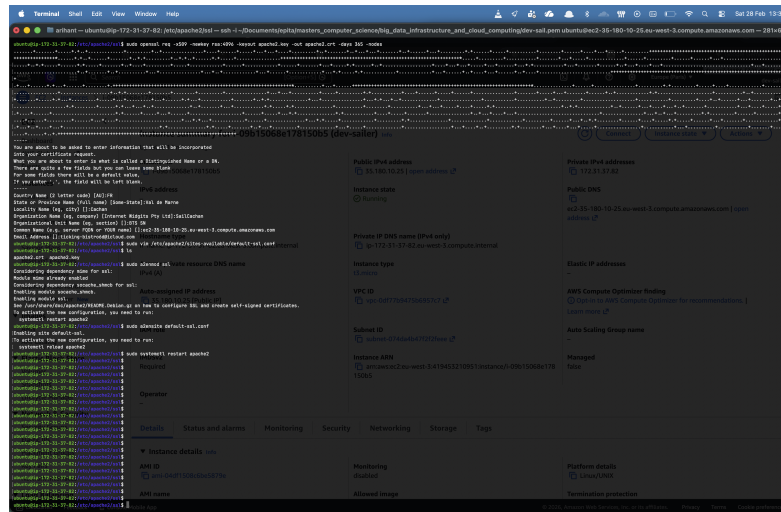


Figure 10: Generating a self-signed SSL certificate (Arihant Jain)

While this enables HTTPS, the browser displays a security warning because the certificate is not issued by a trusted Certificate Authority (CA). The connection is encrypted, but the browser cannot verify the server's identity.

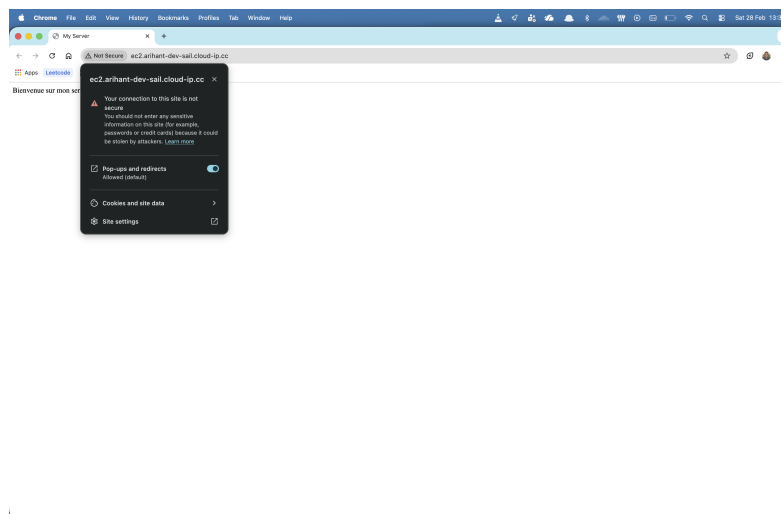


Figure 11: Browser warning for self-signed certificate (Arihant Jain)

## Question 11. Create and Install a Certified Certificate with ZeroSSL. Test with a Web Browser

We explored ZeroSSL as an option for obtaining a trusted certificate. However, ZeroSSL's free tier has limitations — it requires manual renewal every 90 days, the dashboard-based workflow is less straightforward for server-side automation, and domain verification can be cumbersome compared to other solutions. For these reasons, we opted to skip ZeroSSL and proceed directly with Let's Encrypt and Certbot, which offers a fully automated and CLI-based approach better suited for server administration.

## Question 12. Create and Install a Certified Certificate with Let's Encrypt and Certbot. Test with a Web Browser

To obtain a trusted SSL certificate with automatic renewal, we used Certbot with Let's Encrypt. First, we install Certbot and its Apache plugin:

```
sudo apt install certbot python3-certbot-apache
```

Then we request a free SSL certificate for our domain:

```
sudo certbot --apache -d arihant-dev-sail.cloud-ip.cc
```

Certbot automatically configures Apache to use the new certificate.

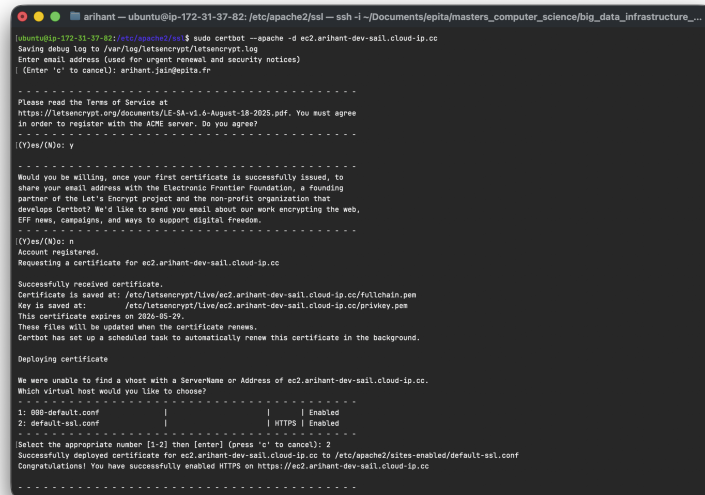


Figure 12: Certbot successfully obtaining a Let's Encrypt certificate (Arihant Jain)

The site is now accessible securely at <https://arihant-dev-sail.cloud-ip.cc> with a valid SSL certificate issued by Let's Encrypt. The browser shows a padlock icon confirming the secure connection.

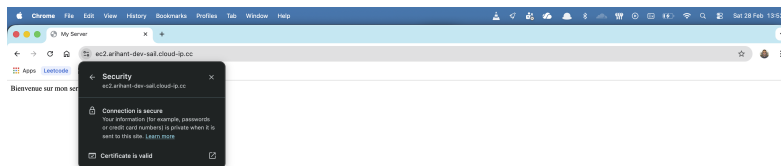


Figure 13: Valid Let's Encrypt certificate in the browser (Arihant Jain)

### Question 13. Enable the Apache Rewrite Engine and Create the HTTP to HTTPS Redirect Rule. Test with a Web Browser

To ensure all traffic is served over HTTPS, we enable the Apache rewrite module and configure an automatic redirect from HTTP to HTTPS:

```
sudo a2enmod rewrite
sudo systemctl restart apache2
```

We then edit the Apache virtual host configuration for port 80:

```
sudo nano /etc/apache2/sites-available/000-default.conf
```

Adding the following rewrite rules inside the `<VirtualHost *:80>` block:

```
<VirtualHost *:80>
    ServerName arihant-dev-sail.cloud-ip.cc
    RewriteEngine On
    RewriteCond %{HTTPS} off
    RewriteRule ^(.*)$ https://%{HTTP_HOST}%{REQUEST_URI} [R=301,L]
</VirtualHost>
```

After restarting Apache, any request to `http://arihant-dev-sail.cloud-ip.cc` is automatically redirected to `https://arihant-dev-sail.cloud-ip.cc`. Note that Certbot can also configure this redirect automatically when prompted during certificate installation.

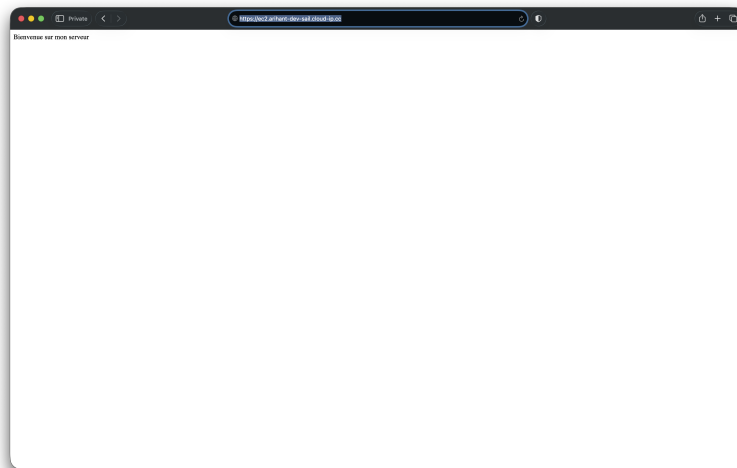


Figure 14: HTTP to HTTPS redirect working in the browser (Arihant Jain)

## Exercise 3. Creating a PHP Application

### Question 14. Install the PHP Module

To serve dynamic content with Apache, we install PHP and the Apache PHP module:

```
sudo apt install php libapache2-mod-php php-curl
sudo systemctl restart apache2
```

PHP is now installed and loaded as an Apache module, allowing the server to process `.php` files.

### Question 15. Create a `phpinfo.php` Script and Identify the PHP Version

To verify that PHP is correctly installed and configured with Apache, we create a test file:

```
sudo nano /var/www/html/phpinfo.php
```

With the following content:

```
<?php
phpinfo();
?>
```

Navigating to `https://arihant-dev-sail.cloud-ip.cc/phpinfo.php` displays the full PHP configuration page. From this page, we can identify the PHP version currently installed on the server.



Figure 15: PHP info page showing the installed PHP version (Arihant Jain)

## Question 16. Create a meteo.php Script That Displays Weather Information Using Open-Weather

To build a weather application, we first set up an OpenWeatherMap API key:

1. Create an account on <https://openweathermap.org>
2. Navigate to the API keys section and generate a new key
3. The API endpoint used is:

`http://api.openweathermap.org/data/2.5/weather?q={city}&units=metric&lang=fr&appid={apiKey}`

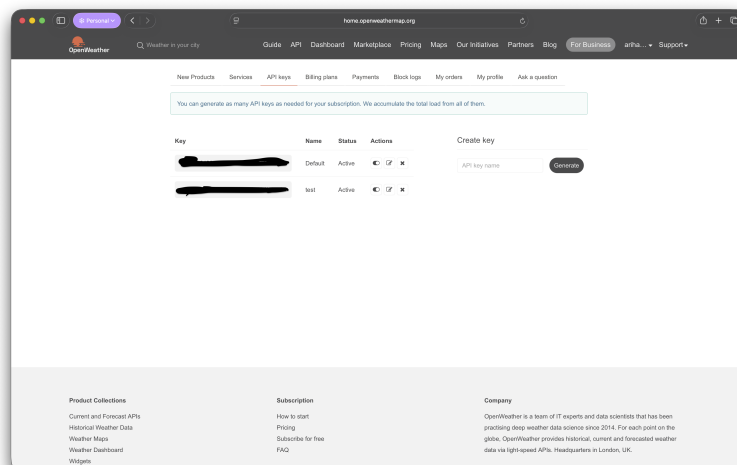


Figure 16: OpenWeatherMap API key dashboard (Arihant Jain)

We then create the weather application file:

```
sudo nano /var/www/html/meteo.php
```

The application fetches weather data for Paris using `file_get_contents()` to call the API and `json_decode()` to parse the JSON response:

```
<?php
$apiKey = "YOUR_API_KEY";
$city = "Paris";

$url = "http://api.openweathermap.org/data/2.5/weather?q="
    . $city . "&units=metric&lang=fr&appid=" . $apiKey;

$response = file_get_contents($url);
$data = json_decode($response, true);

$temperature = $data['main']['temp'];
$description = $data['weather'][0]['description'];
$humidity = $data['main']['humidity'];
$windSpeed = $data['wind']['speed'];
?>

<!DOCTYPE html>
<html>
<head>
    <title>Meteo Paris</title>
</head>
<body>
    <h1>Meteo a <?php echo $city; ?></h1>
    <p>Temperature : <?php echo $temperature; ?> C</p>
    <p>Description : <?php echo $description; ?></p>
    <p>Humidite : <?php echo $humidity; ?>%</p>
    <p>Vent : <?php echo $windSpeed; ?> m/s</p>
</body>
</html>
```

The application is accessible at <https://arihant-dev-sail.cloud-ip.cc/meteo.php>. It calls the OpenWeatherMap API with the city set to Paris, retrieves the current temperature, weather description, humidity, and wind speed, and displays them on the page. The `units=metric` parameter ensures temperatures are in Celsius and `lang=fr` returns descriptions in French.

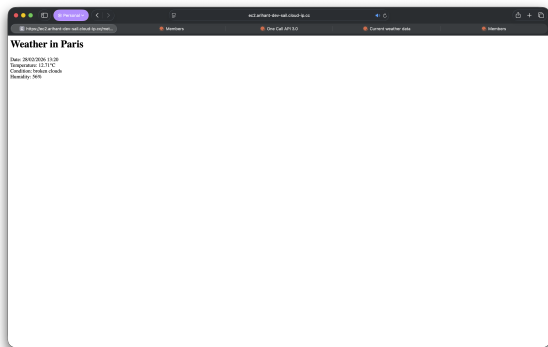


Figure 17: Initial weather page (Arihant Jain)

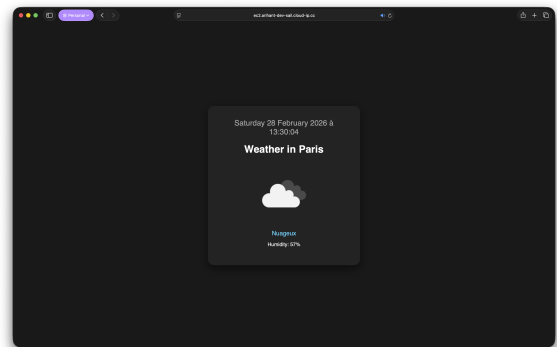


Figure 18: Updated weather page (Arihant Jain)

The left figure shows the initial version of the weather application displaying raw data from the API. The right figure shows the updated version with improved styling and layout for a better user experience.